

## 1. Introduction

The `LinkedList` class in C++ implementation of a singly linked list, one of the fundamental data structures in computer science. Below are `Node.h` file that has `Node` class Definition, and `LinkedList.h` class declaration. (source files are provided on [weonline web site](#))

Our `LinkedList` class provides an interface to create and manipulate linked lists with the following functionalities:

- Adding elements (`push_front`, `push_back`, `insert_at`, `insert_after`)
- Removing elements (`pop_front`, `pop_back`, `erase_at`, `erase`)
- Accessing elements (`front`, `back`, `at`)
- Checking size and emptiness (`size`, `empty`)
- Clearing the list (`clear`)
- Copying lists (copy constructor)
- Comparing lists (equality operator)
- Printing the list contents (overloaded `<<` operator)
- see class declaration for the rest of the methods

`Node.h`

---

```
#pragma once
#include <string>
#include <iostream>
#include <ostream>
using namespace std;
class Node
{
public:
    string data;
    Node* next;
    Node():next(nullptr){}
    Node(const string &na) :next(nullptr) {}

    void print()const {
        cout << data << "->";
    }
    friend ostream& operator<< (ostream & out, const Node & obj);
};

ostream& operator<<(ostream& out, const Node& obj){
    out << obj.data << string("->");
    return out;
}
```

---

`LinkedList.h`

```
#include "Node.h"
#include <string>
#include <iostream>
using namespace std;
class LinkedList {
public:
    // Constructors
    LinkedList();
    LinkedList(const LinkedList& other);
    ~LinkedList();
```

```

// Element Access
string& at(int index);
string& front();
string& back();

// Modifiers
void push_back(const string& value); //or append
void pop_back();
void push_front(const string& value);
void pop_front(); //remove first
void insert_at(int index, const string& value);
void insert_after(int index, const string& val);
void erase_at(int index);
void erase(const string& e);
void clear(); //delete all nodes to make list empty

// Capacity
int size() const;
bool empty() const;

LinkedList& operator=(const LinkedList& lhs);
bool operator==(const LinkedList& lhs) const;

friend ostream& operator<<(ostream& out, const LinkedList &list);
private:
    Node* head;
};

```

---

## 2. Linked List Specification and Explanation

Following Section provides explanation of above `LinkedList` class methods for linked list implementation. Your implementation must follow these explanations.

### Class Methods

#### Constructors:

- `LinkedList();`
  - Default constructor. Initializes an empty linked list (head is set to nullptr).
- `LinkedList(const LinkedList& other);`
  - Copy constructor makes a deep copy of another `LinkedList` object, creating new nodes to clone the original list.
- `~LinkedList();`
  - Destructor. Iterates through the list and frees up memory allocated for each node to prevent memory leaks.

#### Element Access:

- `string& at(int index);`
  - Gets a reference to the element at the specified `index`.
- `string& front();`
  - Gets a reference to the first element in the list.
- `string& back();`
  - Gets a reference to the last element in the list.

#### Modifiers:

- `void push_back(const string& value);`
  - Adds a new node containing `value` to the end of the list.
- `void pop_back();`
  - Removes the last element from the list.
- `void push_front(const string& value);`
  - Adds a new node containing `value` to the beginning of the list.
- `void pop_front();`
  - Removes the first element from the list.
- `void insert_at(int index, const string& value);`
  - Inserts a new node containing `value` at the specified `index`.
- `void insert_after(int index, const string& val);`
  - Finds the node at `index` and inserts a new node containing `val` after it.
- `void erase_at(int index);`
  - Removes the node at the specified `index`.
- `void erase(const string& e);`
  - Removes the first node with the value `e`.
- `void clear();`
  - Removes all elements from the list to clean up memory.

### Capacity:

- `int size() const;`
  - Returns the number of elements in the list.
- `bool empty() const;`
  - Returns `true` if the list is empty, `false` otherwise.

### Operators:

- `LinkedList& operator=(const LinkedList& lhs);`
  - Assignment operator. Performs a deep copy, replacing the contents of the current list with the contents from `lhs`.
- `bool operator==(const LinkedList& lhs) const;`
  - Equality operator. Checks if two linked lists have the same elements in the same order.

### Friend Function:

- `ostream& operator<<(ostream& out, const LinkedList &list);`
  - Provides a way to print the contents of the linked list using the output stream (`out`). This is often used with `cout` to easily print the list.

### 3. Tasks For Lab

- 3.1 Task 1: Implement Linked List Operations: In this task, you will implement various operations for a singly linked list using the provided LinkedList and Node classes.
- 3.2 Task 2: Implementing User Interaction with Linked List Operations. Create a user-friendly interface that allows users to interact with the LinkedList class and perform various operations on a linked list (use following menu to interact with user). Users will be able to add elements, remove elements, access elements, and perform other operations through a menu-driven interface.

```
Welcome to the Linked List Program!

Menu:
1. Add an element
2. Remove an element by value
3. Access an element at index
4. Insert an element at index
5. Insert an element after index
6. Remove an element at index
7. Print the linked list
8. Print the size of the linked list
9. Check if the linked list is empty
10. Clear the linked list
x. Exit
```

- 3.3 Task 3: Finding the Middle Node (Add this item as a menu item as 11<sup>th</sup> entry and test) Design and implement a member function for the LinkedList class that we have implemented in Task1. This method is aimed to find the middle node of a singly linked list. The middle node is defined as the node that is at the middle of the linked list. If the number of nodes in the list is even, there are two middle nodes; in this case, you will return the second middle node. Inside the LinkedList class, implement a function named **findMiddleNode** that returns a pointer to the middle node of the linked list. If the linked list is empty, the function should return nullptr. Test and validate your implementation by various scenarios, including linked lists with different lengths.
- 3.4 Inside the LinkedList class, implement a method named **getSmallestNode** that returns a pointer to the smallest(in our class that will be lexicographically smallest)node in the linked list. Ensure that the method correctly handles an empty linked list by returning nullptr. (Add this item as a menu item as 12<sup>th</sup> entry)
- 3.5 Design and implement a member function (that is inside the LinkedList class) named **moveSmallestToFront** that moves the smallest node to the front of the linked list. Ensure that the method correctly handles cases where the list is empty or contains only one node. (Add this item as a menu item as 13<sup>th</sup> entry)

Upload your project in the form of only .cpp .h files.